

UNIT III CONSTRAINT SATISFACTION PROBLEMS AND GAME THEORY

**JAL1401
PRINCIPLES
OF
ARTIFICIAL
INTELLIGENCE**

UNIT – III CONSTRAINT SATISFACTION PROBLEMS AND GAME THEORY SYLLABUS

Local search for constraint satisfaction problems -
Adversarial search - Games - Optimal decisions and
strategies in games - The minimax search procedure
- Alpha-beta pruning - Additional refinements -
Iterative deepening

CONSTRAINT SATISFACTION PROBLEMS (CSP)

- A problem is solved when each variable has a value that satisfies all the constraints on the variable. A problem described this way is called a constraint satisfaction problem, or CSP.
- CSP → Problems for which finding a solution that meets a set of constraints is the goal.
- Eg. For tasks including resource allocation, planning, scheduling, and decision-making, CSPs are frequently employed in AI.
- Finding values for a group of variables that fulfill a set of restrictions or rules is the aim of constraint satisfaction problems.

CONSTRAINT SATISFACTION PROBLEMS (CSP)

Basic components in the constraint satisfaction problem:

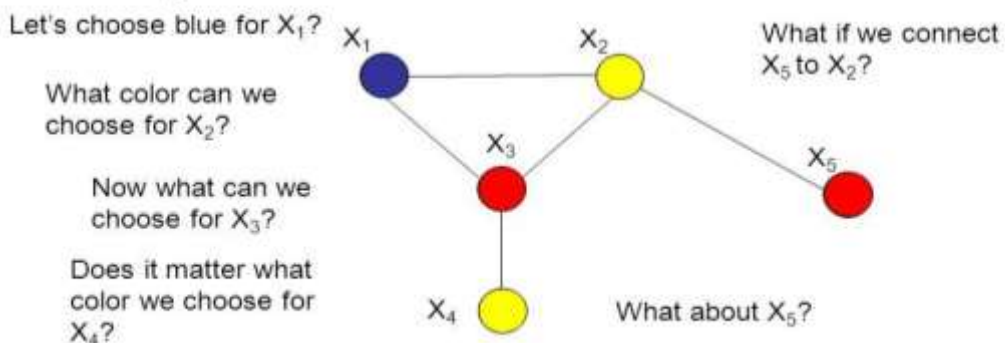
- **Variables:** The things that need to be determined. Eg. squares in a chess game are variables.
- **Domains:** The range of potential values that a variable can have. Eg. the set of chess coins represented as domain in chess game.
- **Constraints:** The guidelines that control how variables relate to one another. Eg. ensuring that the King is not under attack by opponent in chess game.

CONSTRAINT SATISFACTION PROBLEMS (CSP)

PROBLEM	VARIABLES	DOMAIN	CONSTRAINTS
Sudoku Puzzle	Empty cells	Numbers from 1 to 9	Ensuring that no number is repeated in a row, column, or 3x3 subgrid
Scheduling Problems (classes in univ)	Classes	Time slots	Ensuring that classes with overlapping students or instructors cannot be scheduled simultaneously
Map coloring	Regions or countries	Available colors	Ensuring that adjacent regions must have different colors

CONSTRAINT SATISFACTION PROBLEMS (CSP)

- Graph coloring
 - Assume we can use the colors {red, blue, yellow}
 - Can we color the graph such that a each adjacent node has a different color?



CONSTRAINT SATISFACTION PROBLEMS (CSP)

Basic components in the constraint satisfaction problem:

- **Variables:** The things that need to be determined are variables. Variables in a CSP are the objects that must have values assigned to them in order to satisfy a particular set of constraints. Boolean, integer, and categorical variables are just a few examples of the various types of variables. Variables, for instance, could stand in for the many puzzle cells that need to be filled with numbers in a sudoku puzzle.
- **Domains:** The range of potential values that a variable can have is represented by domains. Depending on the issue, a domain may be finite or limitless. For instance, in Sudoku, the set of numbers from 1 to 9 can serve as the domain of a variable representing a problem cell.
- **Constraints:** The guidelines that control how variables relate to one another are known as constraints. Constraints in a CSP define the ranges of possible values for variables. Unary constraints, binary constraints, and higher-order constraints are only a few examples of the various sorts of constraints. For instance, in a sudoku problem, the restrictions might be that each row, column, and 3×3 box can only have one instance of each number from 1 to 9.

CONSTRAINT SATISFACTION PROBLEMS (CSP)

- **Constraint Satisfaction Problems (CSP) representation:**
- The finite set of **variables** $V_1, V_2, V_3, \dots, V_n$.
- Non-empty domain for every single variable $D_1, D_2, D_3, \dots, D_n$.
- The finite set of constraints $C_1, C_2, \dots, C_m$.
 - where each constraint C_i restricts the possible values for variables,
 - e.g., $V_1 \neq V_2$
 - Each constraint C_i is a pair $\langle \text{scope}, \text{relation} \rangle$
 - Example: $\langle (V_1, V_2), V_1 \text{ not equal to } V_2 \rangle$
 - Scope = set of variables that participate in constraint.
 - Relation = list of valid variable value combinations → a clear list of permitted combinations.

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS

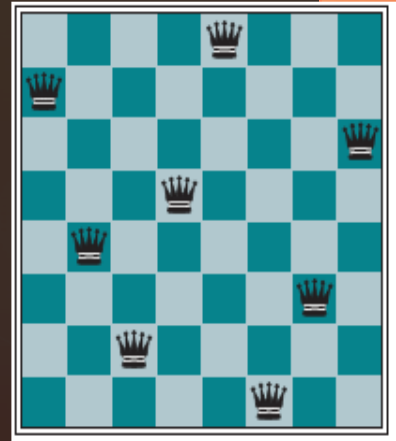
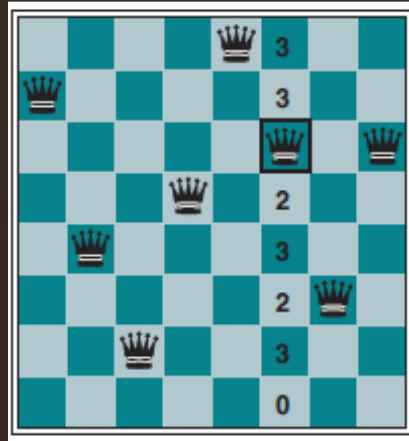
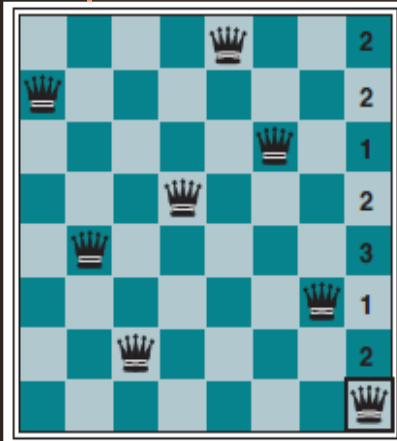
- Local search - effective in solving many CSPs
- 2 techniques: 1. Min-conflicts heuristic 2. Constraint weighting
- Each state assigns a value to every variable, and the search changes the value of one variable at a time.
- Start with a complete assignment to the 8 variables → several constraints violated → randomly choose a conflicted variable (Q8 in the eg.) → change the value to something that leads closer to a solution (approach → select the value that results in the minimum number of conflicts with other variables → the min-conflicts heuristic).

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS

1. Min-conflicts heuristic

- Each state assigns a value to every variable, and the search changes the value of one variable at a time.
- Start with a complete assignment to the 8 variables → several constraints violated → randomly choose a conflicted variable (Q8 in the eg.) → change the value to something that leads closer to a solution (approach → select the value that results in the minimum number of conflicts with other variables → the min-conflicts heuristic).

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS



1 2 3 4 5 6 7 8

1 2 3 4 5 6 7 8

1 2 3 4 5 6 7 8

Min-conflicts heuristic applied

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS

- Min-conflicts local search algorithm for CSPs

```

function MIN-CONFLICTS(csp, max_steps) returns a solution or failure
  inputs: csp, a constraint satisfaction problem
           max_steps, the number of steps allowed before giving up

  current ← an initial complete assignment for csp
  for i = 1 to max_steps do
    if current is a solution for csp then return current
    var ← a randomly chosen conflicted variable from csp.VARIABLES
    value ← the value v for var that minimizes CONFLICTS(csp, var, v, current)
    set var = value in current
  return failure
    
```

- The initial state → chosen randomly or by a greedy assignment process that chooses a minimal-conflict value for each variable in turn.
- The CONFLICTS function → counts the number of constraints violated by a particular value, given the rest of the current assignment.

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS

- Min-conflicts is effective for many CSPs
- The run time of min-conflicts (after the initial assignment) is independent of problem size.
- Works well for easy (like n-queens puzzle) and hard problems (like scheduling) .
- Many local search algorithms (like hill climbing search) can be used with CSP.

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS

- The landscape of a CSP under the min-conflicts heuristic has a series of plateaus.
- Millions of variable assignments that are only one conflict away from a solution possible.
- Plateau search → allowing sideways moves to another state with the same score - can help local search find its way off this plateau.
- Tabu search → keeping a small list of recently visited states and forbidding the algorithm to return to those states.

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS

2. Constraint weighting

- Concentrate the search on the important constraints.
- Each constraint is given a numeric weight, initially all 1.
- At each step of the search, the algorithm chooses a variable/value pair to change that will result in the **lowest total weight** of all violated constraints.
- The weights are then adjusted by incrementing the weight of each constraint that is violated by the current assignment.

LOCAL SEARCH FOR CONSTRAINT SATISFACTION PROBLEMS

- Benefits of Constraint weighting technique:
- It adds topography to plateaus, making sure that it is possible to improve from the current state
- It also adds learning → over time the difficult constraints are assigned higher weights.
- Advantage of local search → can be used in an online setting when the problem changes – eg. repairing a schedule of flights due to bad weather.

ADVERSARIAL SEARCH

- Competitive environments (two or more agents have conflicting goals), give rise to adversarial search problems.
- Board games have simplified nature and lesser (finite) states and actions defined by precise rules.
- Physical games (like croquet and ice hockey) have more complicated descriptions, a larger range of possible actions, and rather imprecise rules defining the legality of actions – so not attractive in AI community



ADVERSARIAL SEARCH

- Multi-agent environments considered in the aspect of
 - ❑ Economy
 - ❑ Nondeterministic environment
 - ❑ Consisting of adversarial agents (performing adversarial game-tree search)
- Games with optimal moves considered
- Pruning → makes the search more efficient by ignoring portions of the search tree
- Heuristic evaluation function → to estimate who is winning based on features of the state

ADVERSARIAL SEARCH

Two-player zero-sum games

- Games most commonly studied within AI → deterministic, two-player, turn-taking, perfect information (fully observable), zero-sum (win-lose) games
- Move → means action; position → means state
- 2 players : Player1 → MAX, Player2 → MIN
- Elements of the game:
 - S_0 – initial state
 - TO-MOVE(s) – player to make next move in state s
 - ACTIONS(s) - The set of legal moves in state s.
 - RESULT(s, a) - The transition model – ACTION(s) → RESULT(s), a is the action
 - IS-TERMINAL(s) - A terminal test, true when the game is over and false otherwise. States where the game has ended are called terminal states)
 - • UTILITY(s, p) - A utility function that defines the final numeric value to player p when the game ends in terminal state s (Chess game – final numeric values are 1, 0, $\frac{1}{2}$)

ADVERSARIAL SEARCH

For a game, the following are maintained:

- **State space graph** → a graph where the vertices are states, the edges are moves and a state might be reached by multiple paths → defined by S_0 , ACTIONS function, and RESULT function
- **Search tree** → tree superimposed over part of the state space graph to determine what move to make.
- **Complete game tree** → a search tree that follows every sequence of moves all the way to a terminal state → may be infinite for an unbounded state space.

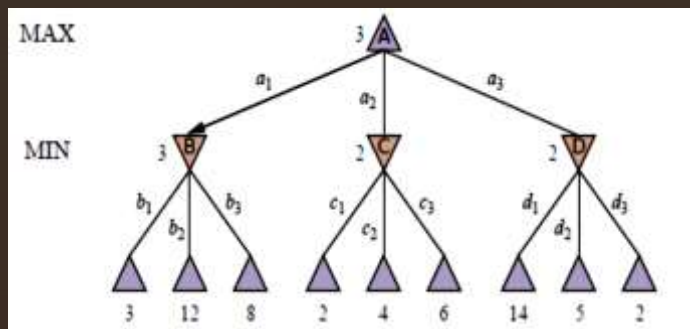
OPTIMAL DECISIONS IN GAMES

- MAX (player 1) wants to find a sequence of actions leading to a win, and MIN tries to oppose it.
- MAX's strategy must be a conditional plan → a contingent (dependent) strategy specifying a response to each of MIN's possible moves.
- Binary outcome – win or lose
- Desirable outcome → winning a game – needs a strategy
- For games with multiple outcome scores (different scores for different outcomes of different actions), minimax search preferred.

OPTIMAL DECISIONS IN GAMES

- Given a game tree, the optimal strategy determined by working out the minimax value of each state in the tree → MINIMAX(s).
- MINIMAX(s) → utility (for MAX) for 2 optimal players.
- For a terminal state → The minimax value is just its utility value (Numerical values assigned to terminal nodes representing the outcome's desirability for the MAX).
- For a nonterminal state → in MAX's turn - MAX prefers to move to a state of maximum value; in MIN's turn – MIN prefers to move to a state of minimum value

OPTIMAL DECISIONS IN GAMES



A 2-ply game tree

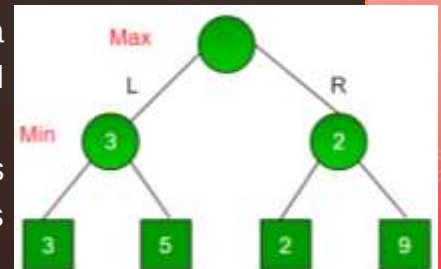
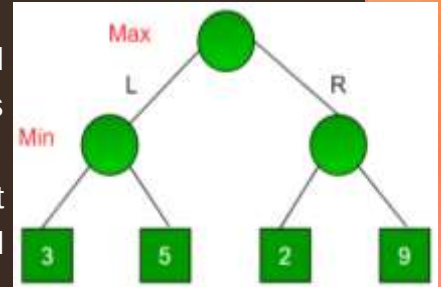
$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if To-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if To-MOVE}(s) = \text{MIN} \end{cases}$$

THE MINIMAX SEARCH PROCEDURE

- Minimax → a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally.
- Used in two player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc. Assumption → both are optimal players.
- In Minimax the two players are called maximizer and minimizer. The maximizer tries to get the highest score possible while the minimizer tries to do the opposite and get the lowest score possible.
- MINIMAX(s) can be computed using the function provided.
- Minimax search algorithm → a search algorithm that finds the best move for MAX by trying all actions and choosing the one whose resulting state has the highest MINIMAX value.
- A recursive algorithm → proceeds down to the leaves and then backs up the minimax values through the tree as the recursion unwinds.

THE MINIMAX SEARCH PROCEDURE

- Every board state has a value associated with it.
- Consider a game which has 4 final states and paths to reach final state are from root to 4 leaves of a perfect binary tree.
- Since this is a backtracking based algorithm, it tries all possible moves, then backtracks and makes a decision.
- If Maximizer goes LEFT, the minimizer now has a choice between 3 and 5. Being the minimizer it will definitely choose the least among both, that is 3.
- If Maximizer goes RIGHT, the minimizer now has a choice between 2 and 9. He will choose 2 as it is the least among the two values.



THE MINIMAX SEARCH PROCEDURE

An algorithm for calculating the optimal move using minimax

function MINIMAX-SEARCH(*game, state*) *returns an action*

player $\leftarrow$ *game*.TO-MOVE(*state*)

value, move $\leftarrow$ MAX-VALUE(*game, state*)

return move

function MAX-VALUE(*game, state*) *returns a (utility, move) pair*

if game.IS-TERMINAL(*state*) *then return* *game*.UTILITY(*state, player*), null

v, move $\leftarrow -\infty$

for each a in game.ACTIONS(*state*) *do*

v2, a2 $\leftarrow$ MIN-VALUE(*game, game*.RESULT(*state, a*))

if v2 > v then

v, move $\leftarrow$ *v2, a*

return v, move

function MIN-VALUE(*game, state*) *returns a (utility, move) pair*

if game.IS-TERMINAL(*state*) *then return* *game*.UTILITY(*state, player*), null

v, move $\leftarrow +\infty$

for each a in game.ACTIONS(*state*) *do*

v2, a2 $\leftarrow$ MAX-VALUE(*game, game*.RESULT(*state, a*))

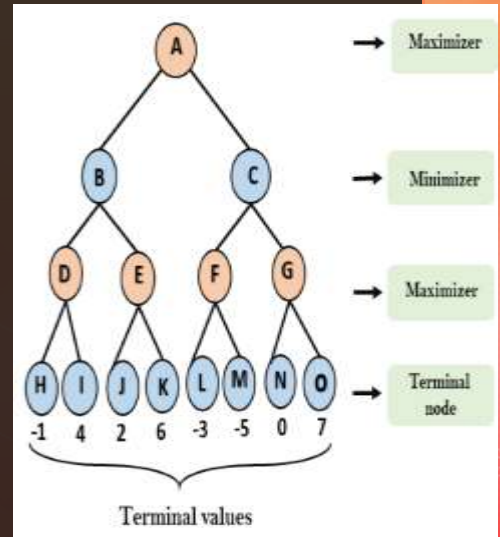
if v2 < v then

v, move $\leftarrow$ *v2, a*

return v, move

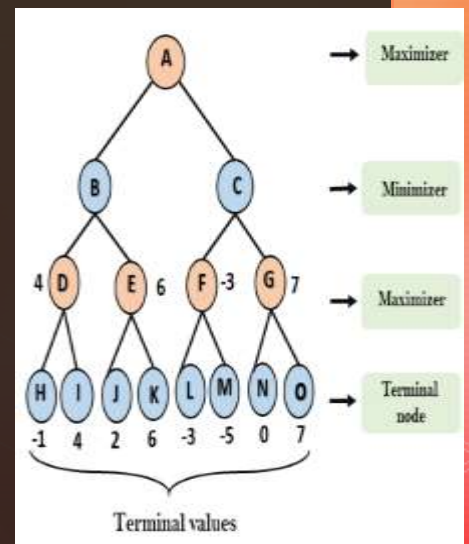
THE MINIMAX SEARCH PROCEDURE

- **Step-1:** In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states.
- In the given tree diagram, let's take A is the initial state of the tree.
- Suppose maximizer takes first turn which has worst-case initial value = $-\infty$, and minimizer will take next turn which has worst-case initial value = $+\infty$.



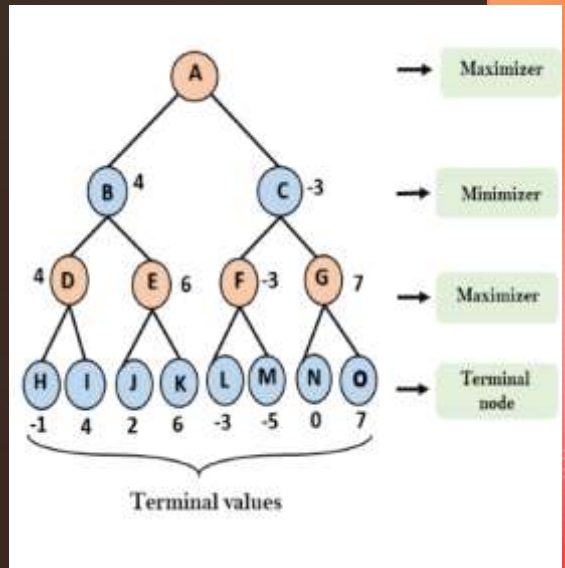
THE MINIMAX SEARCH PROCEDURE

- **Step-2:** It is Maximizer's turn. Find the utilities value for the Maximizer, its initial value is $-\infty$, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values.
- It will find the maximum among the all.
- For node D $\rightarrow \max(-1, -\infty) \rightarrow \max(-1, 4) = 4$
- For Node E $\rightarrow \max(2, -\infty) \rightarrow \max(2, 6) = 6$
- For Node F $\rightarrow \max(-3, -\infty) \rightarrow \max(-3, -5) = -3$
- For node G $\rightarrow \max(0, -\infty) \rightarrow \max(0, 7) = 7$



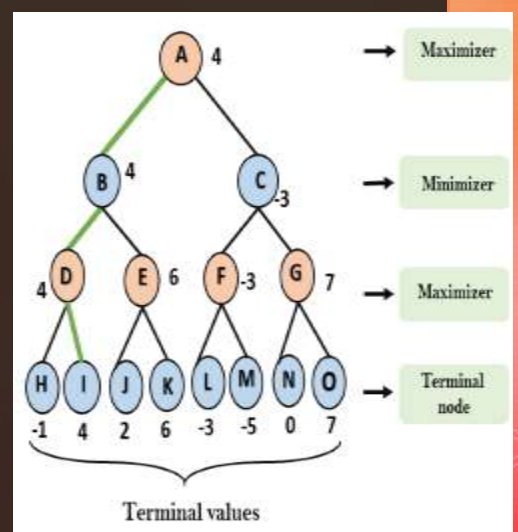
THE MINIMAX SEARCH PROCEDURE

- **Step-3:** It's minimizer's turn, so it will compare all nodes value with $+\infty$, and will find the 3rd layer node values.
- For node B = $\min(4, 6) = 4$
- For node C = $\min(-3, 7) = -3$



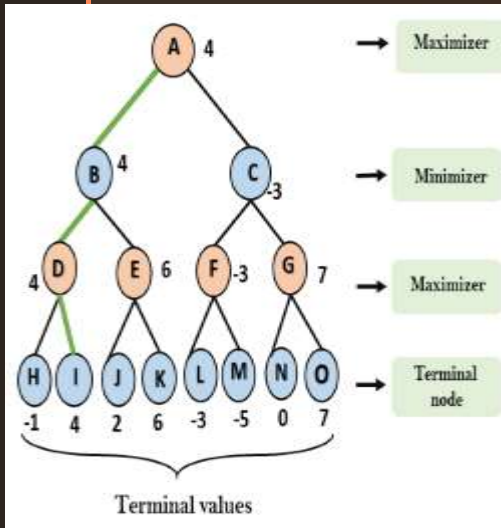
THE MINIMAX SEARCH PROCEDURE

- **Step-4:** It's Maximizer's turn, and it will again choose the maximum of all nodes value and find the maximum value for the root node.
- In this game tree, there are only 4 layers, hence the root node has been reached immediately, but in real games, there will be more number of layers.
- For node A $\max(4, -3) = 4$



THE MINIMAX SEARCH PROCEDURE

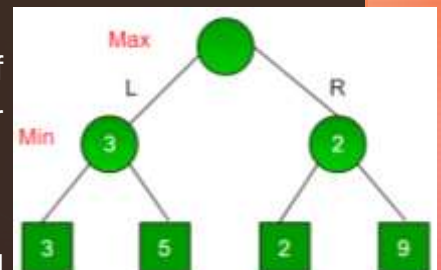
The recursive procedure for finding the minimax value



- $\text{MAX}(B, C)$
- $\rightarrow \text{MAX}(\text{MIN}(D, E), \text{MIN}(F, G))$
- $\rightarrow \text{MAX}(\text{MIN}(\text{MAX}(H, I), \text{MAX}(J, K)), \text{MIN}(\text{MAX}(L, M), \text{MAX}(N, O)))$
- $\rightarrow \text{MAX}(\text{MIN}(4, 6), \text{MIN}(-3, 7))$
- $\rightarrow \text{MAX}(4, -3) \rightarrow 4$

THE MINIMAX SEARCH PROCEDURE

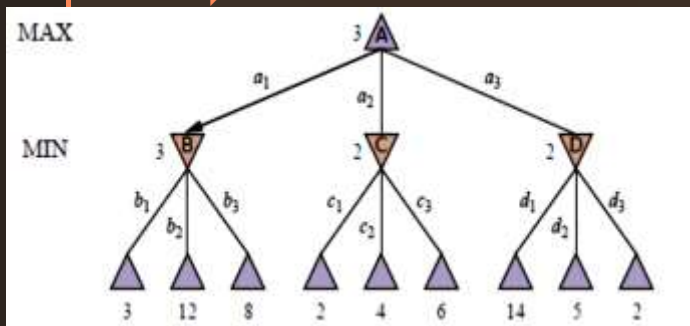
- A complete depth-first exploration of the game tree is performed.
- If $b \rightarrow$ branching factor and $d \rightarrow$ maximum depth of tree
- **Time complexity:** $O(b^d) \rightarrow$ because of exponential complexity, it is impractical for complex games
- **Space complexity:**
 - $O(bd)$, if algorithm that generates all actions at once
 - $O(d)$, if algorithm that generates one action at a time



ALPHA-BETA PRUNING

- Modified version of the minimax algorithm → an optimization technique
- Minimax search alg. is exponential over 'd' → this can be cut to half
- Pruning → a technique by which without checking each node of the game tree the correct minimax decision can be computed → computing the correct minimax decision without examining every state by pruning large parts of the tree that make no difference to the outcome → can be applied at any depth of a tree → can prune the tree leaves and also an entire sub-tree.
- Alpha – beta pruning → the technique that uses pruning and 2 parameters (Alpha and Beta) on Minimax alg.

ALPHA-BETA PRUNING

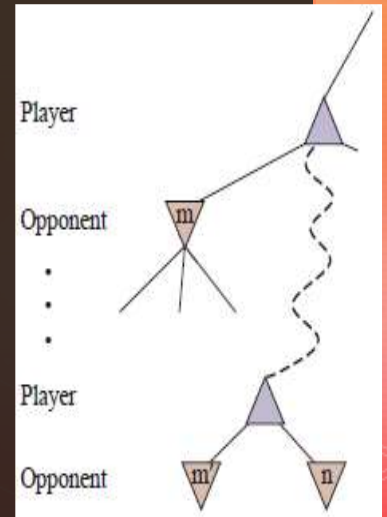


$\text{MINIMAX}(A)$
 $= \max(B, C, D)$
 $= \max(\min(3, 12, 8), \min(2, 4, 6), \min(14, 5, 2))$
 $= \max(3, 2, 2) = 3$

- If actions c_2 and c_3 lead to utility values x & y (instead of 4 & 6):
- $\text{MINIMAX}(A) = \max(B, C, D) = \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2))$
- $= \max(3, \min(2, x, y), 2) = \max(3, z, 2)$ where $z = \min(2, x, y) \leq 2 = 3$

ALPHA-BETA PRUNING

- General principle:
- Consider a node n somewhere in the tree, such that Player has a choice of moving to n .
- If Player has a better choice either at the same level or at any point higher up in the tree, then Player will never move to n .
- So once enough details is found out about n (by examining some of its descendants) to reach this conclusion (moving or not moving to n), it can be pruned.
- Eg. if n is worse than m or m' , n can be pruned.



ALPHA-BETA PRUNING

- 2 parameters used apart from state $s \rightarrow \alpha, \beta$
- α = the value of the best (i.e., highest-value) choice found so far at any choice point along the path for MAX = "at least" = $-\infty$
- β = the value of the best (i.e., lowest-value) choice found so far at any choice point along the path for MIN = "at most" = $+\infty$
- The alg. updates the values of α and β .
- Then prunes the remaining branches at a node as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.
- To prune $\rightarrow$ the algorithm terminates the recursive call $\rightarrow$ occurs when $\alpha \geq \beta$

ALPHA-BETA PRUNING

Condition for Alpha-beta pruning

- Alpha: At any point along the Maximizer path, Alpha is the best option or the highest value we've discovered. The initial value for alpha is $-\infty$.
- Beta: At any point along the Minimizer path, Beta is the best option or the lowest value we've discovered.. The initial value for beta is $+\infty$.
- The condition for Alpha-beta Pruning is that $\alpha \geq \beta$.
- The alpha and beta values of each node must be kept track of. Alpha can only be updated when it's MAX's time, and beta can only be updated when it's MIN's turn.
- MAX will update only alpha values and the MIN player will update only beta values.
- The node values will be passed to upper nodes instead of alpha and beta values during going into the tree's reverse.
- Alpha and Beta values only are passed to child nodes.

ALPHA-BETA PRUNING

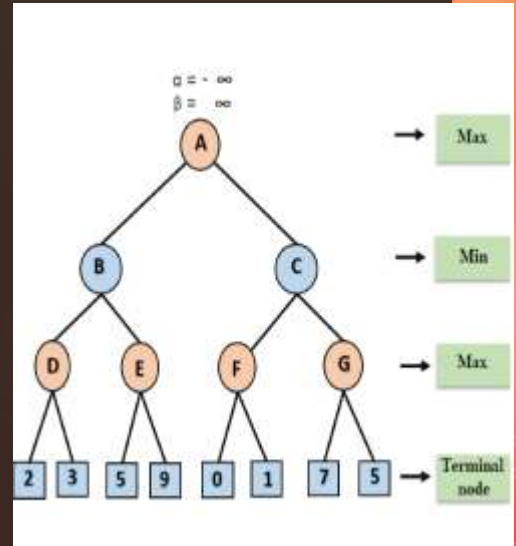
```
function ALPHA-BETA-SEARCH(game, state) returns an action
  player  $\leftarrow$  game.TO-MOVE(state)
  value, move  $\leftarrow$  MAX-VALUE(game, state, - $\infty$ , + $\infty$ )
  return move

function MAX-VALUE(game, state,  $\alpha, \beta$ ) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v  $\leftarrow -\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a),  $\alpha, \beta$ )
    if v2 > v then
      v, move  $\leftarrow$  v2, a
       $\alpha$   $\leftarrow$  MAX( $\alpha, v$ )
    if v  $\geq$   $\beta$  then return v, move
  return v, move

function MIN-VALUE(game, state,  $\alpha, \beta$ ) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v  $\leftarrow +\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a),  $\alpha, \beta$ )
    if v2 < v then
      v, move  $\leftarrow$  v2, a
       $\beta$   $\leftarrow$  MIN( $\beta, v$ )
    if v  $\leq$   $\alpha$  then return v, move
  return v, move
```

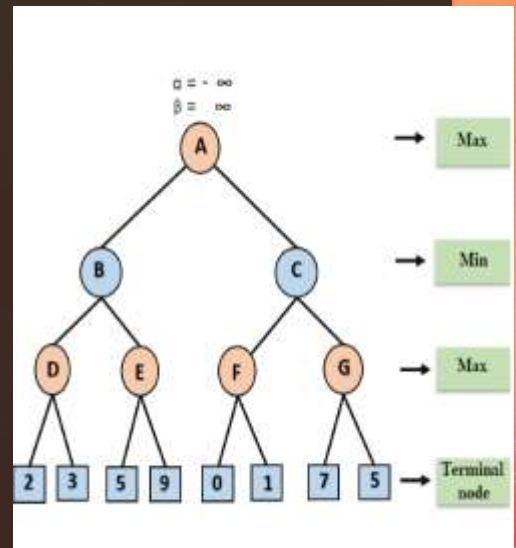
ALPHA-BETA PRUNING

- **Step-1:** At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$,
- These values of alpha and beta passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.



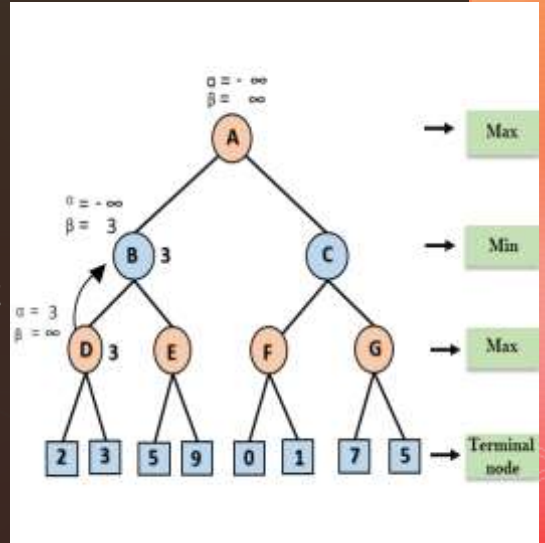
ALPHA-BETA PRUNING

- **Step-2:** At Node D, the value of α will be calculated as its Max's turn. The value of $\alpha = -\infty$ is compared with 2 $\rightarrow \max(-\infty, 2) = 2$ and then 2 is compared with 3, and at node D, $\alpha = \max(2, 3) = 3$



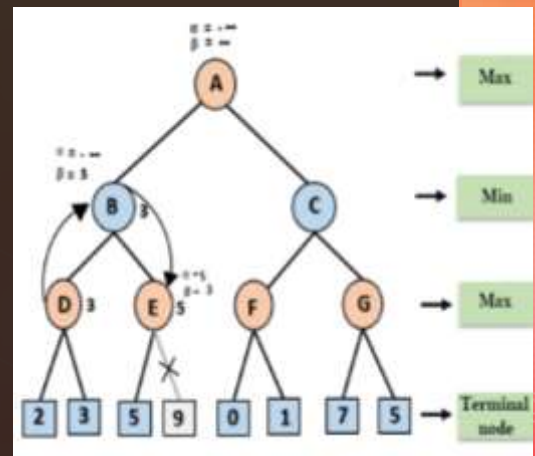
ALPHA-BETA PRUNING

- **Step-3:** Now algorithm backtracks to node B, where the value of β will change as this is Min's turn,
- Now $\beta = +\infty$, will be compared with the available subsequent nodes value, i.e. $\min(\infty, 3) = 3$, hence at node B now $\alpha = -\infty$, and $\beta = 3$.
- In the next step, algorithm traverse the next successor of Node B which is node E, and the values of $\alpha = -\infty$, and $\beta = 3$ will also be passed.



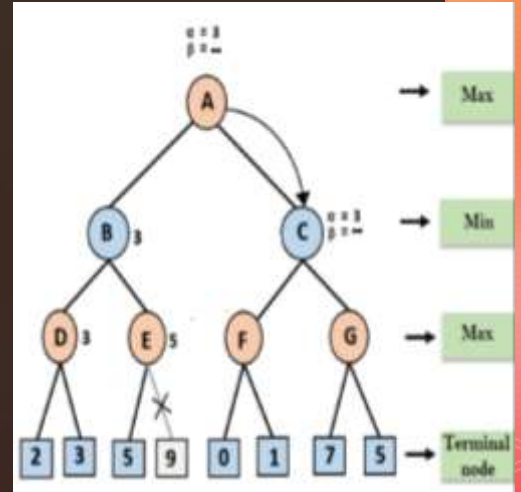
ALPHA-BETA PRUNING

- **Step-4:** At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so $\max(-\infty, 5) = 5$, hence at node E $\alpha = 5$ and $\beta = 3$, where $\alpha \geq \beta$, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



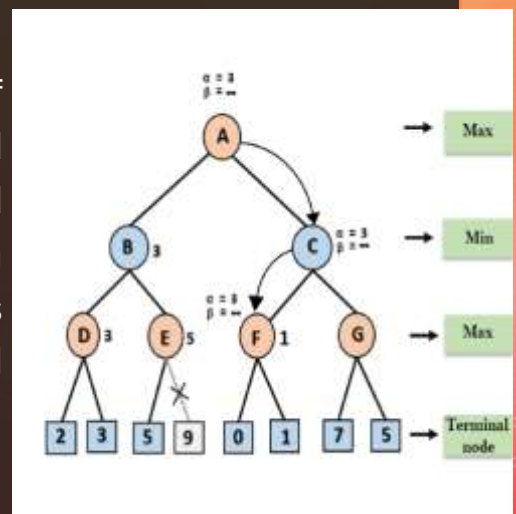
ALPHA-BETA PRUNING

- Step-5:** At next step, algorithm again backtracks the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as $\max(-\infty, 3) = 3$, and $\beta = +\infty$, these two values now pass to right successor of A which is Node C.
- At node C, $\alpha = 3$ and $\beta = +\infty$, and the same values will be passed on to node F.



ALPHA-BETA PRUNING

- Step-6:** At node F, again the value of α will be compared with left child which is 0, and $\max(3, 0) = 3$, and then compared with right child which is 1, and $\max(3, 1) = 3$ still α remains 3, but the node value of F will become 1.



ALPHA-BETA PRUNING

-
- Diagram illustrating a minimax search tree with levels and player types:
- Level 0 (Root):** Node A (Max node, orange circle). Labels: $\alpha = 3$, $\beta = -$.
 - Level 1:** Nodes B (Min node, blue circle) and C (Min node, blue circle). Label for C: $\alpha = 3$, $\beta = 1$.
 - Level 2:** Nodes D (Max node, orange circle), E (Max node, orange circle), F (Max node, orange circle), and G (Max node, orange circle).
 - Level 3 (Terminal nodes):** Blue squares containing values: 2, 3, 5, 9 (under D); 0, 1 (under E); 7, 5 (under F).
 - Pruning:** Indicated by 'X' marks on the branches leading to nodes E and G.
 - Player Labels:** Max (Level 0 and 2), Min (Level 1), Terminal node (Level 3).

ALPHA-BETA PRUNING

-
- The diagram shows a minimax search tree. The root node is A (Max), which has children B and C (Min). B has children D and E (Max), and C has children F and G (Max). D has children 2 and 3 (Terminal nodes), E has children 5 and 9 (Terminal nodes), F has children 0 and 1 (Terminal nodes), and G has children 7 and 5 (Terminal nodes). The path A-B-D-3 is highlighted in green, indicating the optimal path. Pruning is shown with 'X' marks on the branches to E, G, and the right child of G.

ALPHA-BETA PRUNING

NODE	TURN	STEP	VALUE AT NODE	α	β	$\alpha \geq \beta$
At A	Max's			$-\infty$	$+\infty$	False
At B	Min's	Values passed from A		$-\infty$	$+\infty$	False
At D	Max's	Values passed from B		$-\infty$	$+\infty$	False
At D		$\alpha = \text{Max}(\alpha, 2) = \text{Max}(-\infty, 2) = 2$	2	2	$+\infty$	False
At D		$\alpha = \text{Max}(\alpha, 3) = \text{Max}(2, 3) = 3$	3	3	$+\infty$	False
At D			$\text{Max}(2, 3) = 3$			
At B	Min's	Gets 3 from D	3	$-\infty$	3	False
At E	Max's	Values passed from B		$-\infty$	3	False
At E		$\alpha = \text{Max}(\alpha, 5) = \text{Max}(-\infty, 5) = 5$	5	5	3	True
At E		PRUNE THE NEXT SUCCESSOR	5			

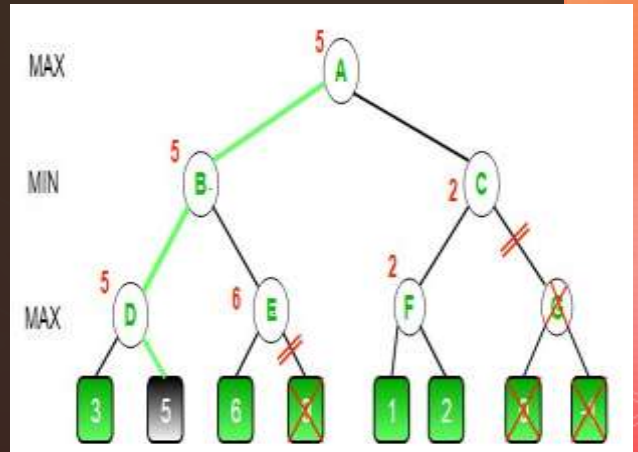
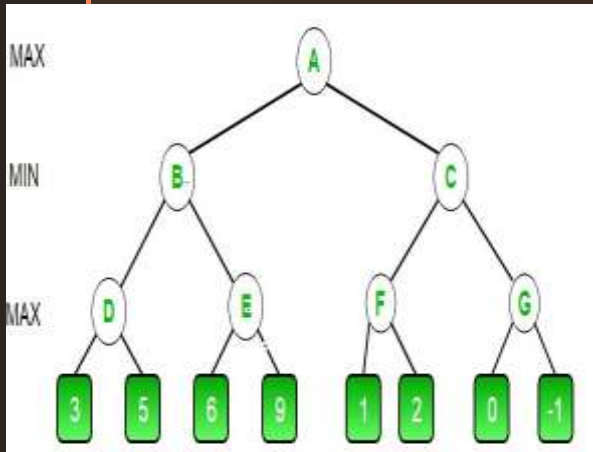
ALPHA-BETA PRUNING

NODE	TURN	STEP	VALUE AT NODE	α	β	$\alpha \geq \beta$
At B	Min's	Gets 5 from E	$\text{Min}(3, 5) = 3$			
At A	Max's	Gets 3 from B	3	3	$+\infty$	False
At C	Min's	Values passed from A		3	$+\infty$	False
At F	Max's	Values passed from C		3	$+\infty$	False
At F		$\alpha = \text{Max}(\alpha, 0) = \text{Max}(3, 0) = 3$	0	3	$+\infty$	False
At F		$\alpha = \text{Max}(\alpha, 1) = \text{Max}(3, 1) = 3$	1	3	$+\infty$	False
At F			$\text{Max}(0, 1) = 1$			
At C	Min's	Gets 1 from F		3	1	True
At C	Min's	PRUNE THE NEXT SUCCESSOR	1	3	1	True
At A	Max's	Gets 1 from C	$\text{Max}(3, 1) = 3$			

ALPHA-BETA PRUNING

Example 2:

Solution: Search path



ALPHA-BETA PRUNING

- The initial call starts from **A**. The value of alpha here is **-INFINITY** and the value of beta is **+INFINITY**. These values are passed down to subsequent nodes in the tree. At **A** the maximizer must choose max of **B** and **C**, so **A** calls **B** first
- At **B** it the minimizer must choose min of **D** and **E** and hence calls **D** first.
- At **D**, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at **D** is $\max(-\text{INF}, 3)$ which is 3.
- To decide whether its worth looking at its right node or not, it checks the condition $\beta \leq \alpha$. This is false since $\beta = +\text{INF}$ and $\alpha = 3$. So it continues the search.
- D** now looks at its right child which returns a value of 5. At **D**, $\alpha = \max(3, 5)$ which is 5. Now the value of node **D** is 5

ALPHA-BETA PRUNING

- **D** returns a value of 5 to **B**. At **B**, $\beta = \min(+\infty, 5)$ which is 5. The minimizer is now guaranteed a value of 5 or lesser. **B** now calls **E** to see if he can get a lower value than 5.
- At **E** the values of alpha and beta is not $-\infty$ and $+\infty$ but instead $-\infty$ and 5 respectively, because the value of beta was changed at **B** and that is what **B** passed down to **E**
- Now **E** looks at its left child which is 6. At **E**, $\alpha = \max(-\infty, 6)$ which is 6. Here the condition becomes true. β is 5 and α is 6. So $\beta \leq \alpha$ is true. Hence it breaks and **E** returns 6 to **B**

ALPHA-BETA PRUNING

- Note how it did not matter what the value of **E**'s right child is. It could have been $+\infty$ or $-\infty$, it still wouldn't matter, We never even had to look at it because the minimizer was guaranteed a value of 5 or lesser. So as soon as the maximizer saw the 6 he knew the minimizer would never come this way because he can get a 5 on the left side of **B**. This way we didn't have to look at that 9 and hence saved computation time.
- **E** returns a value of 6 to **B**. At **B**, $\beta = \min(5, 6)$ which is 5. The value of node **B** is also 5
- The 9 is crossed out because it was never computed.
- **B** returns 5 to **A**. At **A**, $\alpha = \max(-\infty, 5)$ which is 5. Now the maximizer is guaranteed a value of 5 or greater. **A** now calls **C** to see if it can get a higher value than 5.
- At **C**, $\alpha = 5$ and $\beta = +\infty$. **C** calls **F**
- At **F**, $\alpha = 5$ and $\beta = +\infty$. **F** looks at its left child which is a 1. $\alpha = \max(5, 1)$ which is still 5.

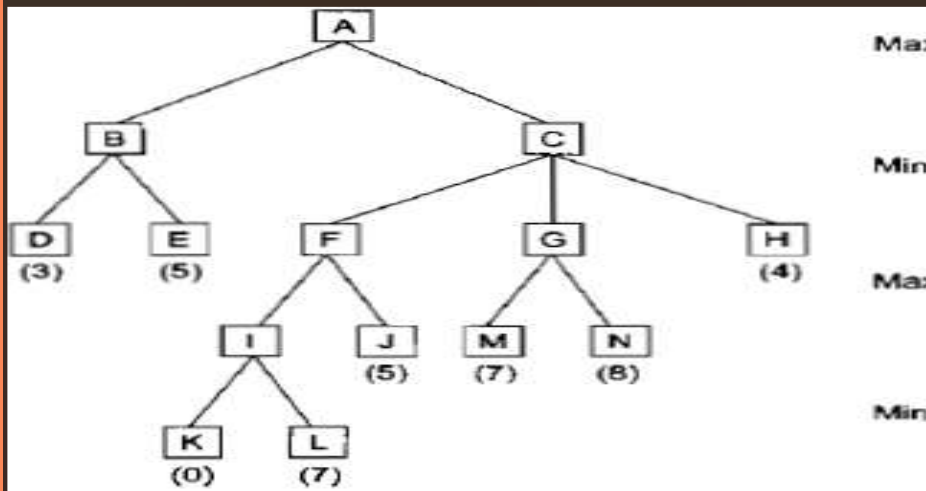
ALPHA-BETA PRUNING

- **F** looks at its right child which is a 2. Hence the best value of this node is 2. Alpha still remains 5
- **F** returns a value of 2 to **C**. At **C**, $\beta = \min(+\text{INF}, 2)$. The condition $\beta \leq \alpha$ becomes true as $\beta = 2$ and $\alpha = 5$. So it breaks and it does not even have to compute the entire sub-tree of **G**.
- The intuition behind this break-off is that, at **C** the minimizer was guaranteed a value of 2 or lesser. But the maximizer was already guaranteed a value of 5 if he choose **B**. So why would the maximizer ever choose **C** and get a value less than 2 ? Again you can see that it did not matter what those last 2 values were. We also saved a lot of computation by skipping a whole sub-tree.
- **C** now returns a value of 2 to **A**. Therefore the best value at **A** is $\max(5, 2)$ which is a 5.
- Hence the optimal value that the maximizer can get is 5

ALPHA-BETA PRUNING

Example 3

:



ALPHA-BETA PRUNING

- The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined.
- 2 types of move order:
- **Worst ordering:** when no leaves are pruned in the tree.
- **Ideal ordering:** when lots of pruning happens in the tree, and best moves occur at the left side of the tree.

ADDITIONAL REFINEMENTS

- 4 refinements done to MINIMAX search procedure
 1. Waiting for Quiescence
 2. Secondary search
 3. Using book moves
 4. Alternatives to minimax

ADDITIONAL REFINEMENTS

1. Waiting for Quiescence (quietness / stability)

- A concept used in game playing and search algorithms within the field of AI.
- Quiescence → a state in which the position of the game is relatively stable and has limited tactical or forcing moves.
- Starting at the leaf nodes of the main search, at certain nodes in the search tree, the algorithm might pause the search and evaluate the position more deeply if it is not yet quiet.
- It's logical to try to get the best of both worlds by combining a dumb search with a smart selective search for quiescence

ADDITIONAL REFINEMENTS

2. Secondary search

- Searching in a path for hidden pitfalls – so that they do not exist a few moves further away than the original search explored.
- This is to make sure that the further moves in a branch are still good.

ADDITIONAL REFINEMENTS

3. Using Book Moves

- Looking at the catalogue and extracting the correct move for a current game configuration, is feasible for some games
- Eg. the ending and opening moves in a chess game have a general style which can be put in a catalogue.
- List of possible moves is termed as book moves
- Enhances the performance of a program

ADDITIONAL REFINEMENTS

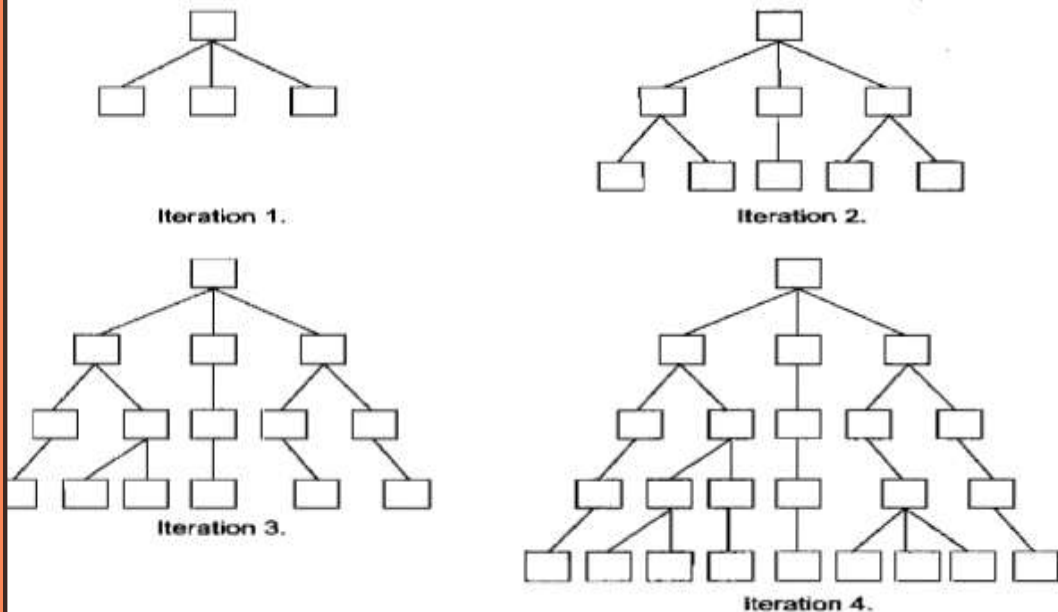
4. Alternatives to minimax

- Assumption in minimax → opponent being an optimal player
- Accepted in winning situation, but not in a losing situation.
- A solution → Take a risk (may be a bad move) that will be advantageous if the opponent would make a mistake.

ITERATIVE DEEPENING SEARCH (IDS)

- Used originally in Chess 4.5
- Single ply search → searching in the next one level – by applying its static evaluation function to the result of each of its possible moves.
- 2-ply search → searching for a depth of 2
- Iterative deepening search → On each iteration the tree is searched one level deeper.
- Can be used with uninformed and informed searches.

ITERATIVE DEEPENING SEARCH (IDS)



ITERATIVE DEEPENING SEARCH (IDS)

- A wasteful approach of searching more levels
- But previous iterations may be helpful
- Game-playing programs are time-constrained – so fixed-depth search may consume more time
- But with IDS, search can be aborted at any time and the best move identified in the previous iteration can be played.
- The path with superior moves, identified in one iteration can be searched first in the next iteration
- If used with alpha-beta pruning with effective ordering, many branches can be pruned and searching time can be drastically reduced.

ITERATIVE DEEPENING SEARCH (IDS)

- DFS → space efficient, but requires cutoff depth at some level
- BFS → not space efficient, but can find the shortest path solution
- DFID → depth-first iterative deepening → dfs + bfs aspects

Algorithm: Depth-First Iterative Deepening

1. Set SEARCH-DEPTH = 1.
2. Conduct a depth-first search to a depth of SEARCH-DEPTH. If a solution path is found, then return it.
3. Otherwise, increment SEARCH-DEPTH by 1 and go to step 2.

- Space efficiency → amount of memory proportional to the no. of nodes in solution path.
- Only the nth iteration is useful - Work spent in the nth iteration is equivalent to work spent in 1st to (n-1)th iterations → in a complete tree
- DFS is uninformed search – but DFID is an optimal one (in terms of time & space)

ITERATIVE DEEPENING SEARCH (IDS)

Advantages of IDS

- IDS gives us the hope to find the solution if it exists in the tree.
- When the solutions are found at the lower depths say n , then the algorithm proves to be efficient and in time.
- IDS search operation tries to improve the depth definition, heuristics, and scores of searching nodes so as to enable efficiency in the search algorithm.
- It has quick responsiveness. The early results indications are a plus point in this algorithm. This followed up with multiple refinements after the individual iteration is completed.
- The performance of IDS is better than single BFS and DFS operating exclusively.
- Space and time complexities are expressed as: $O(d)$ where d is defined as goal depth.

ITERATIVE DEEPENING SEARCH (IDS)

Disadvantages of IDS

- The time taken is exponential to reach the goal node.
- The main problem with IDS is the time and wasted calculations that take place at each depth.
- The situation is not as bad as we may think of especially when the branching factor is found to be high.
- The IDS might fail when the BFS fails. When we are to find multiple answers from the IDDFS, it gives back the success nodes and its path once even if it needs to be found again after multiple iterations.